

Introduction to Multiple Joins in Queries

Associativity vs Cost Implications

Uni Auckland Databases 51

Gerald Weber

Associativity of Joins

- ▶ Joins are associative:

$$(R \bowtie S) \bowtie T = R \bowtie (S \bowtie T)$$

- ▶ This holds for both natural joins and theta joins.

Schema Example: Relation Schema with N-to-M Association

- ▶ **Entities:**

- ▶ Students(StudentID, Name, Age)
- ▶ Courses(CourseID, CourseName, Credits)

- ▶ **Association:**

- ▶ Enrollments(StudentID, CourseID, EnrollmentDate)

Query Example 1

$\pi_{\text{Students.Name, Courses.CourseName}}$

$\sigma_{\text{Courses.CourseName} = \text{'Database Systems'}}$

$\sigma_{\text{Enrollments.EnrollmentDate} \leq \text{'2024-12-31'}}$

$(\text{Students} \bowtie_{\text{Students.StudentID} = \text{Enrollments.StudentID}} \text{Enrollments}$
 $\bowtie_{\text{Enrollments.CourseID} = \text{Courses.CourseID}} \text{Courses})$

Indexing Strategy

Choosing Indexes:

▶ Indexes to Consider:

- ▶ Clustered index:
 $I_c(\text{Enrollments.CourseID}, \text{Enrollments.EnrollmentDate})$
- ▶ Clustered index: $I_c(\text{Courses.CourseID})$
- ▶ Non-clustered index: $I_{nc}(\text{Courses.CourseName})$
- ▶ Clustered index: $I_c(\text{Students.StudentID})$

▶ Rationale:

- ▶ Index on StudentID in Students and CourseID in Courses for quick lookups.
- ▶ The Composite index is good for the given question
- ▶ We execute the join with Courses first.

Query Example 1 optimized

$\pi_{\text{Students.Name, Courses.CourseName}}$
 $(\text{Students} \bowtie_{\text{Students.StudentID} = \text{Enrollments.StudentID}}$
 $\sigma_{\text{Enrollments.EnrollmentDate} \leq '2024-12-31'}(\text{Enrollments}$
 $\bowtie_{\text{Enrollments.CourseID} = \text{Courses.CourseID}}$
 $\sigma_{\text{Courses.CourseName} = 'Database Systems'} \text{Courses}))$

Query Cost Computation (Query 1)

Assumptions:

- ▶ Number of students: 10,000
- ▶ Number of courses: 500
- ▶ Average enrollments per student: 8
- ▶ Access unit cost: 0.1ms

Cost Calculation:

- ▶ **Step 1:** Cost use index on Courses for 'Database Systems': 0.1ms
- ▶ **Step 2:** Cost to join Enrollments with Courses: $10,000 * 0.1\text{ms} = 1,000\text{ms}$
- ▶ **Step 3:** Cost to join Students with Enrollments: $100 * 0.1\text{ms} = 0.01 \text{ seconds}$
- ▶ **Total Cost:** $0.1\text{ms} + 10\text{ms} = 10.1 \text{ ms}$

Query Example 2

$\pi_{\text{Students.Name, Courses.CourseName}}$

$\sigma_{\text{Students.StudentID} = 654321}$

$\sigma_{\text{Enrollments.EnrollmentDate} \leq '2024-12-31'}$

$(\text{Students} \bowtie_{\text{Students.StudentID} = \text{Enrollments.StudentID}} \text{Enrollments}$
 $\bowtie_{\text{Enrollments.CourseID} = \text{Courses.CourseID}} \text{Courses})$

Indexing Strategy

Choosing Indexes:

► Indexes to Consider:

- Clustered index:
 $I_c(\text{Enrollments.CourseID}, \text{Enrollments.EnrollmentDate})$
- Clustered index: $I_c(\text{Courses.CourseID})$
- Clustered index: $I_c(\text{Students.StudentID})$
- Non-clustered index: $I_{nc}(\text{Students.Name})$

► Rationale:

- Non-clustered is enough since we expect only 8 enrollments.
- We execute the join with Students first.

Query Example 2 optimized

$\pi_{\text{Students.Name, Courses.CourseName}}$

$\sigma_{\text{Enrollments.EnrollmentDate} \leq '2024-12-31'}$

$(\sigma_{\text{Students.StudentID} = 654321}$

$\text{Students} \bowtie_{\text{Students.StudentID} = \text{Enrollments.StudentID}} \text{Enrollments})$

$\bowtie_{\text{Enrollments.CourseID} = \text{Courses.CourseID}} \text{Courses}$

Query Cost Computation (Query 2)

Assumptions:

- ▶ Number of students: 10,000
- ▶ Number of courses: 500
- ▶ Average enrollments per student: 8
- ▶ Access unit cost: 0.1ms

Cost Calculation:

- ▶ **Step 1:** Cost use index on Students for 54321: 0.1ms
- ▶ **Step 2:** Cost to join Enrollments with Students: $8 * 0.1\text{ms} = 0.8 \text{ ms}$
- ▶ **Step 3:** Cost to join Courses with Enrollments: $8 * 0.1\text{ms} = 0.8 \text{ ms}$
- ▶ **Total Cost:** $0.1\text{ms} + 2 * 0.8 \text{ ms} = 2 \text{ ms}$

Database Autotuning

Key Aspects of Autotuning:

- ▶ **Index Management:**
 - ▶ Automatic index creation and removal
- ▶ **Performance Monitoring and Analysis:**
 - ▶ Continuous monitoring
 - ▶ Tuning recommendations
- ▶ **Resource Management:**
 - ▶ Safe and reliable operations

Practitioners' Views:

- ▶ **Positive:**
 - ▶ Significant time savings
 - ▶ Continuous optimization
 - ▶ Ease of use for less experienced teams
- ▶ **Challenges:**
 - ▶ Lack of granular control
 - ▶ Trust and reliability concerns
 - ▶ Complexity of implementation

Conclusion

- ▶ Associativity of joins means we can choose the order of executing joins.
- ▶ Cost considerations will often force us to choose a specific order per query.
- ▶ Importance of indexing in query optimization.
- ▶ If we need more than one index, cost considerations can help us choose which is the clustered index.